



Programmiersvorkurs

Einführung in Java – Tag 3

Inhaltsübersicht



Tag 1	Was ist Java? Variablen, Datentypen, (Konvertierung), Kommentare, Livedemo
Tag 2	Boolsche Ausdrücke, If-Abfragen, (Switch-Case)
Tag 3	Arrays, Schleifen, Debugging
Tag 4	Methoden, Klassen, Objektorientierung

Arrays

- Speichert viele Variablen vom gleichen Typ
- Ähneln einem Regal
- Jeder Wert hat sein eigenes „Fach“



Arrays: Beispiel

```
1 int zahl1 = 1;
2 int zahl2 = 1;
3 int zahl3 = 4;
4 int zahl4 = 7;
5 int zahl5 = 19;
6 String hersteller1 = "Mercedes";
7 String hersteller2 = "BMW";
8 String hersteller3 = "Renault";
9 String hersteller4 = "Honda";
```



```
1 zahlenfolge = {1, 1, 4, 7, 19}
2 autohersteller = {"Mercedes", "BMW", "Renault", "Honda"};
```

Wie erstellt man Arrays?



- Typ + eckige Klammern
- In den geschweiften Klammern schreibt die Werte

```
1 int[] zahlenfolge = {1, 1, 4, 7, 19};  
2 String[] autohersteller = {"Mercedes", "BMW", "Renault", "Honda"};
```

Zugriff auf Werte in Arrays

- Zugriff auf Wert über einen Index
- Indexe in Java fangen bei 0 an
- Zu großer Index führt zu einem Fehler

```
1 String[] autohersteller = {"Mercedes", "BMW", "Renault", "Honda"};  
2 String meistenAutosProduziert = autohersteller[3];
```

Index: 0

Index: 1

Index: 2

Index: 3

„**meistenAutosProduziert**“ nimmt den Wert von **autohersteller[]** an der Stelle **3** an, also das 4. Element: „**Honda**“

Vereinfachung

Fach: 0

Fach: 1

Fach: 2

Fach: 3

Fach: 4



„Katze“



„Hund“



„Hamster“



„Fisch“



„Schildkröte“

*Bilder sind nur symbolisch hier, das Regal soll ein „String“ Array darstellen

In Java:

```
String[] haustiere = {"Katze", "Hund", "Hamster", "Fisch", "Schildkröte"};
```

Schleifen



- Wiederholt einen Programmabschnitt
- Wiederholung bricht ab, wenn die **Schleifen-Bedingung** nicht mehr zutrifft

Damit lässt sich z.B. sowas hier in wenigen Zeilen schreiben:

```
1 int countdown = 10;
2
3 System.out.println(countdown); //10
4 countdown--;
5 System.out.println(countdown); //9
6 countdown--;
7 System.out.println(countdown); //8
8
9 //...
10
11 countdown--;
12 System.out.println(countdown); //0
13
14 System.out.println("BOOOOOM");
```


Schleifentypen



for-Schleife

- Für bestimmte Anzahl von Wiederholungen

while-Schleife

- Wiederholungsanzahl nicht fest

While-Schleifen

- An booleschen Wert gebunden (so wie bei „if“)
- Wiederholung läuft so lange, wie der Wert von **Bedingung true** ist

```
1 while (Bedingung) {  
2     Anweisung1;  
3     Anweisung2;  
4     //...  
5 }  
6
```

Wdh.: Boolean

- Aussagen mit Wahrheitswert: **true/false**
- Vergleiche/Verknüpfungen bilden neue Aussagen

Beispiel:

```
1 boolean isHungry = true;
2 boolean hasEaten = false;
3
4 if (isHungry && !hasEaten) {
5     eatSomething();
6     hasEaten = true;
7 }
```

```
A == B
A && B
A || B
!A
!B
```

Für Werte mit
Größenordnung:

```
A == B
A >= B
A <= B
A < B
A > B
```

While-Beispiel



```
boolean istHungrig = true;
```

```
while(istHungrig == true) {  
    //essen oder so  
}
```

```
System.out.println("nicht mehr hungrig");
```

For-Schleifen

- Wird nur eine **bestimmte Anzahl** an Wiederholungen durchgeführt
- Die Klammer nach dem „for“ kann in **3 Bereiche** aufgeteilt werden

```
1 for (int i = 0; i < Anzahl; i++;) {  
2     Anweisung1;  
3     Anweisung2;  
4     //...  
5 }
```

For-Schleifen

Zähler wird
angelegt

Zähler wird
überprüft

Zähler wird
erhöht

```
1 for (int i = 0; i < Anzahl; i++;) {  
2     Anweisung1;  
3     Anweisung2;  
4     //...  
5 }
```

Was genau macht der Code hier?



```
1 int treesAmount = 10;  
2  
3 for (int i = 0; i < treesAmount; i++) {  
4     System.out.println(i + " trees planted!");  
5 }  
6  
7 System.out.println("all done!");
```

Beispiel

Anzahl der
Schleifen-
durchläufe



```
1 int treesAmount = 10;
2
3 for (int i = 0; i < treesAmount; i++) {
4     System.out.println(i + " trees planted!");
5 }
6
7 System.out.println("all done!");
```


Beispiel

Schleifenbeginn



```
1 int treesAmount = 10;
2
3 for (int i = 0; i < treesAmount; i++) {
4     System.out.println(i + " trees planted!");
5 }
6
7 System.out.println("all done!");
```

Beispiel

Zähler *i* und
Text werden
ausgegeben



```
1 int treesAmount = 10;
2
3 for (int i = 0; i < treesAmount; i++) {
4     System.out.println(i + " trees planted!");
5 }
6
7 System.out.println("all done!");
```

Beispiel

Schleife startet
neu

9x



```
1 int treesAmount = 10;
2
3 for (int i = 0; i < treesAmount; i++) {
4     System.out.println(i + " trees planted!");
5 }
6
7 System.out.println("all done!");
```

Beispiel

Finale Ausgabe
nach der Schleife

```
1 int treesAmount = 10;
2
3 for (int i = 0; i < treesAmount; i++) {
4     System.out.println(i + " trees planted!");
5 }
6
7 System.out.println("all done!");
```

Arrays + Schleifen

- Wenn man ein Array durchgeht, erhöht sich der Index um 1

Index: 0

Index: 1

Index: 2

Index: 3

Index: 4

```
1 String[] haustiere = {"Katze", "Hund", "Hamster", "Fisch", "Schildkröte"};  
2 String bestesHaustier = haustiere[0];
```

Die 0 ist hier
der Index!

Arrays + Schleifen

- Wenn man ein Array durchgeht, erhöht sich der Index um 1

Bsp.: jedes Element im Array ausgeben

```
1 String[] haustiere = {"Katze", "Hund", "Hamster", "Fisch", "Schildkröte"};
2 System.out.println(haustiere[0]);
3 System.out.println(haustiere[1]);
4 System.out.println(haustiere[2]);
5 System.out.println(haustiere[3]);
6 System.out.println(haustiere[4]);
```

Arrays + Schleifen

- Bei normalen for-Schleifen erhöht sich der Zähler `i` bei jeder Wiederholung auch um 1

```
1 int treesAmount = 10;
2
3 for (int i = 0; i < treesAmount; i++) {
4     System.out.println(i + " trees planted!");
5 }
6
7 System.out.println("all done!");
```

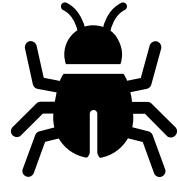
Arrays + Schleifen

- „i“ ist jeweils einmal 0, 1, 2, 3, 4
- Mit `haustiere[i]` wird hier jedes Fach des Arrays einmal besucht

```
1 String[] haustiere = {"Katze", "Hund", "Hamster", "Fisch", "Schildkröte"};  
2  
3 for (int i = 0; i < haustiere.length; i++) {  
4     System.out.println(haustiere[i]);  
5 }
```

`haustiere.length` ist die
Länge des Arrays: 5

Debugging



- Wichtiges Tool für Fehlersuche
- Schrittweises Durchgehen durch ein Programm
- Zeigt Zustand des Programms zu ausgewählten Zeitpunkten an
- Funktioniert etwas unterschiedlich je nach Idea

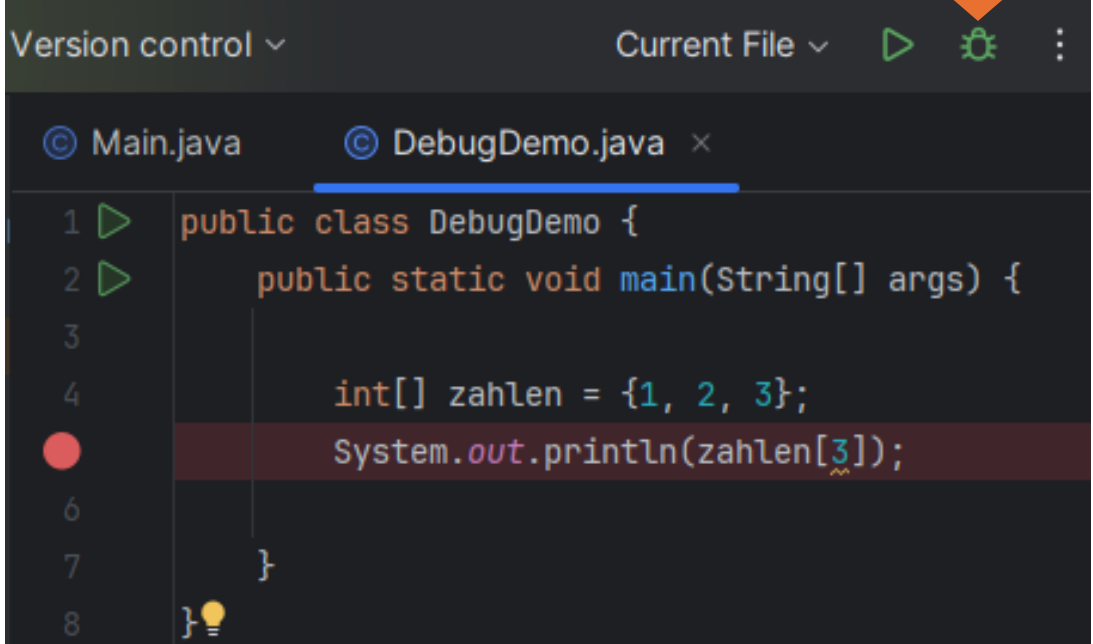
Debugging - Breakpoints

- Roter Punkt (Breakpoint) markiert Zeile bei der angehalten wird
- Setzen durch klicken auf Zeilennummer

```
1 ▶ public class DebugDemo {  
2 ▶     public static void main(String[] args) {  
3           
4         int[] zahlen = {1, 2, 3};  
5         System.out.println(zahlen[3]);  
6     }  
7 }  
8 }  
9 }
```

Debugging - Debug-Modus

- „Käfer“ ist meist das Symbol für den Debug-Modus
- Startet das Programm und läuft bis zum ersten Breakpoint



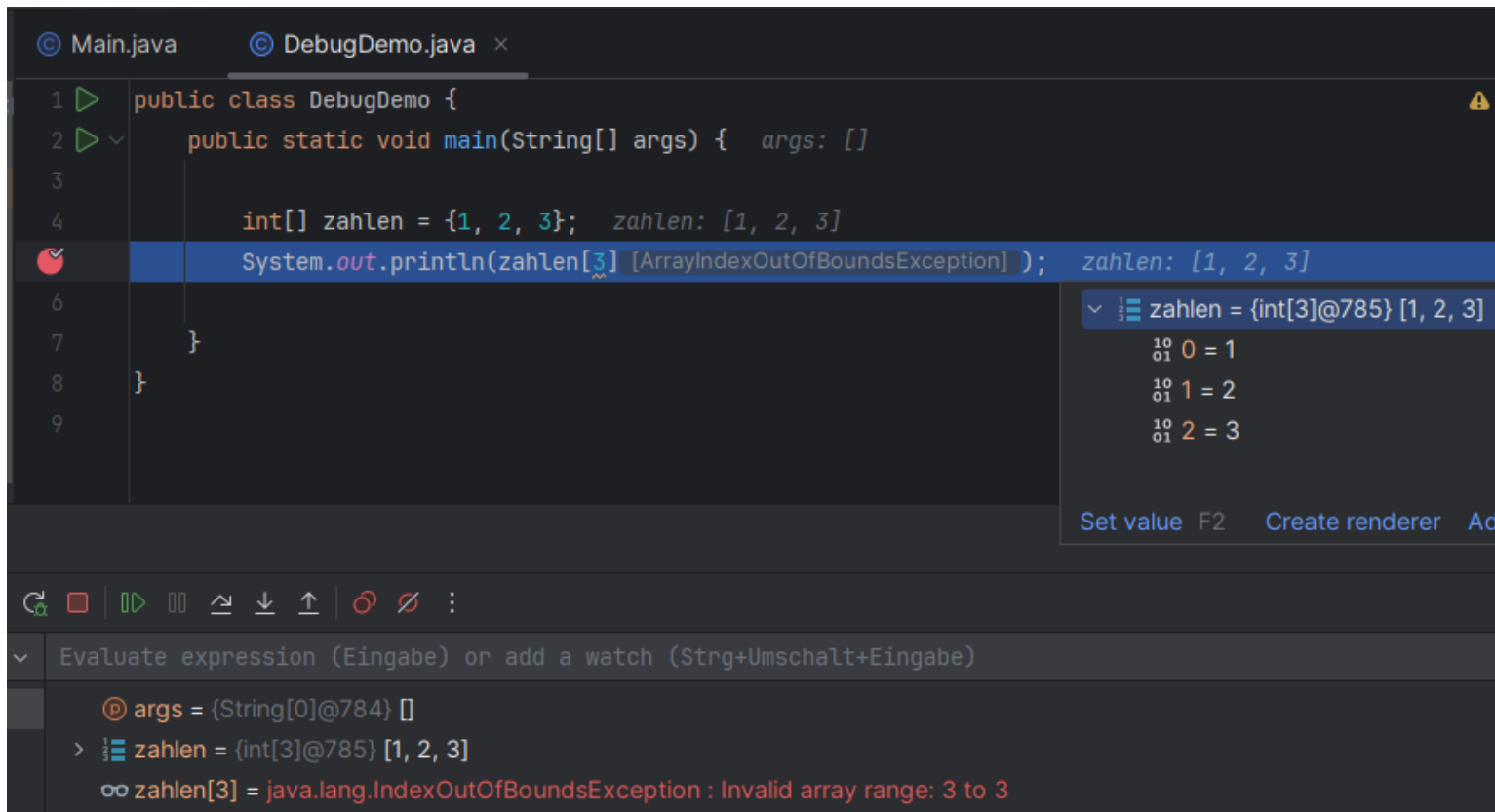
The screenshot shows an IDE interface with two tabs: "Main.java" and "DebugDemo.java". The "DebugDemo.java" tab is active. The code is as follows:

```
1 public class DebugDemo {  
2     public static void main(String[] args) {  
3  
4         int[] zahlen = {1, 2, 3};  
5         System.out.println(zahlen[3]);  
6  
7     }  
8 }
```

A red circle breakpoint is set on line 5. The IDE's top bar shows "Version control" and "Current File" dropdowns, along with a green play button and a green bug icon (the "Käfer" symbol). An orange arrow points down to the bug icon.

Debugging - Debug-Modus

- Werte zu diesem Zeitpunkt werden angezeigt
- Code interaktiv durch hovern/klicken



```
© Main.java  © DebugDemo.java x
1  public class DebugDemo {
2  public static void main(String[] args) {  args: []
3
4      int[] zahlen = {1, 2, 3};  zahlen: [1, 2, 3]
5  System.out.println(zahlen[3] [ArrayIndexOutOfBoundsException]);  zahlen: [1, 2, 3]
6
7  }
8
9  }
```

▼ **zahlen = {int[3]@785} [1, 2, 3]**

10	01	0 = 1
10	01	1 = 2
10	01	2 = 3

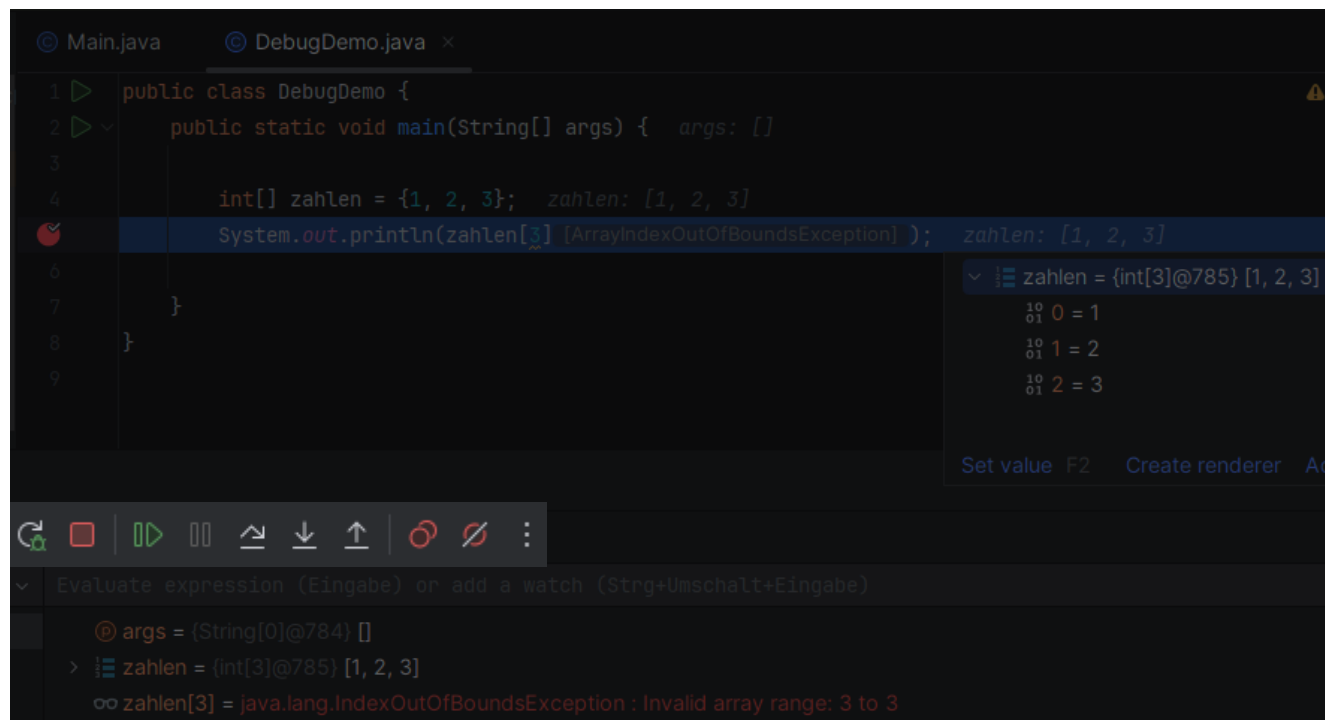
Set value F2 Create renderer Add

Evaluate expression (Eingabe) or add a watch (Strg+Umschalt+Eingabe)

- ① args = {String[0]@784} []
- > zahlen = {int[3]@785} [1, 2, 3]
- zahlen[3] = java.lang.IndexOutOfBoundsException: Invalid array range: 3 to 3

Debugging - Debug-Modus

- Weiterlaufen lassen (bis zum nächsten Breakpoint): „Resume Program“ 
- Nur einen Schritt Weitergehen: „Step Into“ 



The screenshot shows an IDE with two tabs: Main.java and DebugDemo.java. The code in DebugDemo.java is as follows:

```
1 public class DebugDemo {  
2     public static void main(String[] args) { args: []  
3  
4         int[] zahlen = {1, 2, 3}; zahlen: [1, 2, 3]  
5         System.out.println(zahlen[3] [ArrayIndexOutOfBoundsException] ); zahlen: [1, 2, 3]  
6  
7     }  
8 }  
9
```

A breakpoint is set at line 5. The debugger window on the right shows the state of the program:

- zahlen = {int[3]@785} [1, 2, 3]
- 0 = 1
- 1 = 2
- 2 = 3

The bottom of the screenshot shows the debugger's command line and the current state of the program:

- args = (String[0]@784) []
- zahlen = (int[3]@785) [1, 2, 3]
- zahlen[3] = java.lang.IndexOutOfBoundsException : Invalid array range: 3 to 3

Debugging

Zusammenfassung:

1. Breakpoints setzen
2. Im Debugging-Modus starten
3. Schrittweise Programm durchgehen

